

«Sistema de Gestão de EPIs»

Alexandre Chibeloski Bail
Bruno Martins Guimarães
Diego Lúcio Bojarczuk Fioravanti
Gustavo Aparecido Machado

Composição do sistema

- **Backend em C;**
- **Banco de Dados em SQL via SQLite;**
- **GUI e Web Scrapping via Python;**



Composição do sistema

<BACKEND EM C>

- Núcleo original do sistema;
- Gerencia a entrada e filtra os dados;
- Constitui o tipo e a estrutura dos dados;

```
struct EPI {  
    int id;  
    int CA;  
    int QNTD;  
    int QNTD_ALOCADA;  
    char DESCRICAO[100];  
};  
  
static const char *VALIDOS[] = {  
    "capacete", "luva", "bota", "cinto de seguranca",  
    "mangote", "oculos", "protetor auricular", NULL  
};  
  
int filtro_descricao(const char *desc) {  
    for (int i = 0; VALIDOS[i] != NULL; i++)  
        if (strcmp(desc, VALIDOS[i]) == 0) return 1;  
    return 0;  
}
```


Composição do sistema

< SQLite >

- Banco de Dados integrado ao programa
- Existe como um arquivo único, não demanda servidor;
- Funciona como o elo entre o Backend em C e a GUI

em Python;

```
CREATE TABLE ca (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  descricao   TEXT      NOT NULL,
  ca          INTEGER NOT NULL,
  qntd        INTEGER NOT NULL CHECK(qntd >= 0),
  qntd_alocada INTEGER NOT NULL DEFAULT 0,
  ca_validade TEXT      DEFAULT NULL, -- data de vencimento do CA
  ca_status   TEXT      DEFAULT NULL -- OK | VENCIDO | EXPIRANDO | INEXISTENTE
);
```

```
CREATE TABLE emprestimos (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  nome_funcionario TEXT      NOT NULL,
  horario         TEXT      NOT NULL,
  ca_alocado      INTEGER NOT NULL
);
```


Composição do sistema

< GUI em Python >

- Interface gráfica como substituição do terminal;
- Baseada na biblioteca gráfica padrão Tkinter;
- Exibe as informações de forma visual para o usuário;

```
class EPIApp(tk.Tk):          # EPIApp É uma janela
    def __init__(self):
        super().__init__()   # inicializa a janela
        self.title("Gerenciador de EPIs")
        self.geometry("950x660")

        init_db()              # garante que o banco existe
        self._setup_styles()
        self._build_ui()       # constrói todos os widgets
        self.refresh()         # carrega os dados
        self._auto_refresh()# atualiza a cada 2 segundos
```


Composição do sistema

< Web Scraping >

- Extração automatizada de dados de sites;
- BeautifulSoup para leitura de HTML;
- Utilizado quando não se tem a API oficial do site;
- Utilizado para a aquisição das datas de validade dos EPIs e filtro de entrada;

```
lines = full_text.splitlines()
ca_found = False
for i, line in enumerate(lines):
    if re.search(r"n[°o]?s*ca\s*?:?s*$", line.strip().lower()):
        for j in range(i + 1, min(i + 4, len(lines))):
            if lines[j].strip() == ca_str:
                ca_found = True
                break
            elif lines[j].strip():
                break
        if ca_found:
            break
if not ca_found:
    return {"found": False, "error": f"CA {ca_number} nao encontrado."}
```

```
for i, line in enumerate(lines):
    if "validade" in line.strip().lower():
        # Tenta na mesma linha primeiro
        m = re.search(r"\d{2}/\d{2}/\d{4}", line)
        if m:
            result["validade"] = m.group()
        else:
            # Procura nas próximas linhas
            for j in range(i + 1, min(i + 4, len(lines))):
                m = re.search(r"\d{2}/\d{2}/\d{4}", lines[j])
                if m:
                    result["validade"] = m.group()
                    break
```




FIM